



UNIVERSITÀ DEGLI STUDI DI VERONA

Dipartimento di Scienze e Ingegneria

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

**Studio e implementazione di un sistema di comunicazione basato
su modulazione di segnali analogici**

Relazione descrittiva del progetto

Relatore:
Prof. Francesco Visentin

Laureanda:
Chiara Baldo
VR500878

ANNO ACCADEMICO 2025–2026

Indice

1	Teoria di riferimento e obiettivi progettuali	1
1.1	Obiettivo generale del progetto	1
1.2	Architettura generale del sistema	2
1.3	Acquisizione analogica tramite Arduino	2
1.4	Concetto di campionamento	3
1.5	Frequenza di campionamento e asse della FFT	4
1.6	FFT e spettro positivo	4
1.7	Onde quadre e armoniche	5
1.8	Magnitude nell'intorno della frequenza	6
1.9	Ricostruzione tramite box filter	7
2	Implementazione e risoluzione dei problemi	8
2.1	Organizzazione del codice	8
2.2	File script.py: flusso principale	8
2.3	Stima della frequenza di campionamento	9
2.3.1	Perché non usare i timestamp lato PC per ogni campione	10
2.4	File fft.py: calcolo dello spettro positivo	10
2.5	Magnitude nell'intorno della frequenza	11
2.6	Individuazione delle componenti principali	11
2.6.1	Soppressione delle armoniche	12
2.6.2	Gestione delle finestre non affidabili	13
2.7	Ricostruzione delle componenti	14
2.8	File plotting.py	14
2.9	File varie.py	15
3	Testing e risultati ottenuti	16
3.1	Validazione su segnali simulati	16
3.2	Test live con potenziometri	16
3.3	Risultato finale	17

Capitolo 1

Teoria di riferimento e obiettivi progettuali

1.1 Obiettivo generale del progetto

L'obiettivo del progetto è stato lo sviluppo di un sistema hardware/software in grado di acquisire un segnale analogico generato da una board elettronica, trasferirlo a un computer tramite Arduino e analizzarlo in Python mediante tecniche di elaborazione dei segnali.

Il lavoro si inserisce nel contesto dello studio di un sistema di comunicazione basato su segnali analogici modulati o combinati elettronicamente. In particolare, il sistema finale deve essere in grado di ricevere un unico segnale analogico misto, contenente il contributo di due componenti, e separarne le informazioni principali tramite analisi in frequenza.

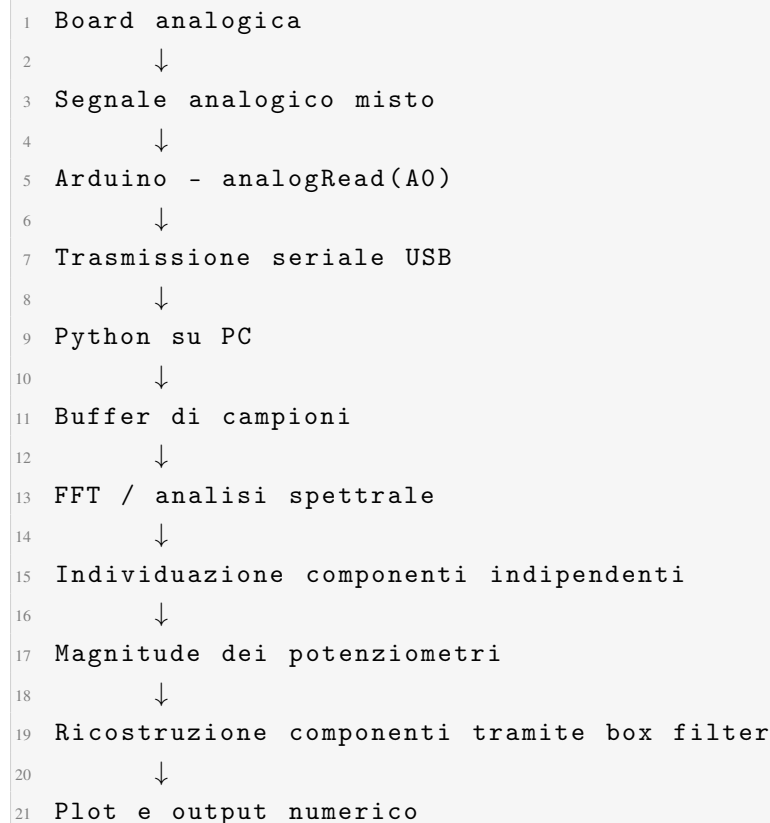
Nel caso finale, la board produce un segnale unico, acquisito da Arduino tramite `'analogRead()'` e inviato al PC su porta seriale. Su PC, il segnale viene raccolto in finestre di campioni e analizzato tramite FFT. L'obiettivo pratico è stimare il valore dei due potenziometri presenti nel circuito osservando la magnitude delle componenti spettrali associate alle due onde presenti nel segnale.

Il sistema realizzato quindi svolge quattro funzioni principali:

1. acquisizione del segnale analogico tramite Arduino;
2. trasferimento dei campioni al PC via seriale;
3. analisi del segnale tramite FFT/short-FFT;
4. stima dei contributi dei due potenziometri tramite magnitude delle componenti spettrali.

1.2 Architettura generale del sistema

L'architettura del progetto può essere schematizzata così:



La board genera un segnale analogico. Arduino non interpreta il segnale in modo complesso: il suo compito è principalmente quello di effettuare una conversione analogico-digitale e inviare i valori al PC. La parte più importante dell'elaborazione è svolta in Python.

Il codice principale usa un buffer circolare tramite `deque(maxlen=BUFFER_SIZE)`, in modo da mantenere sempre gli ultimi 1500 campioni acquisiti. Quando il buffer è pieno e sono arrivati 1500 nuovi campioni rispetto all'ultima FFT, viene calcolata una nuova analisi.

1.3 Acquisizione analogica tramite Arduino

Arduino legge il segnale tramite un pin analogico, nello specifico **A0**.

Lo sketch Arduino ha una struttura essenziale:

```
1 void setup() {
2     Serial.begin(115200);
3 }
4
```

```
5 void loop() {  
6   int x = analogRead(A0);  
7   Serial.println(x);  
8 }
```

Ogni lettura produce un valore intero compreso tra 0 e 1023, poiché `analogRead()` su Arduino restituisce il risultato della conversione ADC a 10 bit.

Il valore letto rappresenta il livello istantaneo del segnale analogico. Questo valore viene inviato al PC come riga di testo. Il PC riceve quindi una sequenza del tipo:

```
1 512  
2 514  
3 511  
4 508  
5 ...
```

Questi valori non sono già centrati intorno allo zero: sono valori ADC positivi. Per questo motivo, prima dell'analisi FFT, il codice Python sottrae la media del segnale, eliminando la componente continua.

1.4 Concetto di campionamento

Per poter analizzare un segnale analogico su un computer, è necessario campionarlo. Il campionamento consiste nel misurare il valore del segnale a intervalli temporali discreti.

Se si acquisiscono N campioni con frequenza di campionamento f_s , allora ogni campione rappresenta un istante separato dal precedente da:

$$T_s = \frac{1}{f_s}$$

dove:

- T_s è il periodo di campionamento;
 - f_s è la frequenza di campionamento.
-

1.5 Frequenza di campionamento e asse della FFT

La FFT restituisce dei coefficienti associati a bin frequenziali. La relazione tra indice del bin e frequenza è:

$$f_k = k \cdot \frac{f_s}{N}$$

dove:

- f_k è la frequenza associata al bin k ;
- f_s è la frequenza di campionamento;
- N è il numero di campioni della finestra;
- k è l'indice del bin.

La risoluzione frequenziale della FFT è:

$$\Delta f = \frac{f_s}{N}$$

Nel progetto finale:

$$f_s = 2445 \text{ Hz}$$

e:

$$N = 1500$$

quindi:

$$\Delta f = \frac{2445}{1500} \approx 1.63 \text{ Hz}$$

Questo significa che la FFT distingue le frequenze con passo di circa 1.63 Hz. Per questo, nel live, le frequenze non compaiono sempre esattamente come 18 Hz o 40 Hz, ma possono apparire come 17.93 Hz, 19.56 Hz, 40.75 Hz o 42.38 Hz. Questi valori sono coerenti con la risoluzione frequenziale e con le naturali variazioni del segnale acquisito.

1.6 FFT e spettro positivo

La Fast Fourier Transform consente di trasformare un segnale dal dominio del tempo al dominio della frequenza. Invece di osservare come il segnale varia campione per campione, si osserva quali frequenze lo compongono.

Per un segnale discreto $x[n]$, la trasformata discreta di Fourier è:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

Il risultato $X[k]$ è, in generale, un numero complesso. Per ogni frequenza abbiamo quindi:

$$X[k] = \text{Re}(X[k]) + j\text{Im}(X[k])$$

La magnitude è:

$$|X[k]| = \sqrt{\text{Re}(X[k])^2 + \text{Im}(X[k])^2}$$

Nel progetto, la magnitude viene interpretata come misura dell'intensità della componente spettrale associata a una data frequenza. Questo è il punto centrale per la stima dei potenziometri.

Poiché il segnale acquisito è reale, lo spettro della FFT è simmetrico. Per questo motivo, nel codice si usa `rfft()`, che restituisce solo la parte positiva dello spettro. La funzione `positive_spectrum()` restituisce infatti le frequenze positive tramite `rfftfreq()` e le ampiezze normalizzate monolato tramite:

```
1 amp = 2 * np.abs(X) / N
```

Inoltre il bin a 0 Hz viene azzerato esplicitamente, così da ignorare la componente continua.

1.7 Onde quadre e armoniche

Una parte importante del progetto riguarda il fatto che i segnali siano onde quadre, che contengono quindi una frequenza fondamentale e una serie di armoniche dispari.

Idealmente, un'onda quadra può essere rappresentata tramite serie di Fourier come:

$$x(t) = \frac{4A}{\pi} \left(\sin(2\pi ft) + \frac{1}{3} \sin(2\pi 3ft) + \frac{1}{5} \sin(2\pi 5ft) + \frac{1}{7} \sin(2\pi 7ft) + \dots \right)$$

Quindi nello spettro di una quadra compaiono componenti a:

$$f, 3f, 5f, 7f, \dots$$

Questo ha creato una criticità importante durante lo sviluppo: se si cercano semplicemente i due picchi più grandi nello spettro, si rischia di selezionare una frequenza fondamentale e una sua armonica, invece delle due componenti indipendenti realmente presenti nel segnale.

Esempio:

```
1 26.5 Hz e 79.5 Hz
```

possono sembrare due frequenze diverse, ma:

$$79.5 = 3 \cdot 26.5$$

Quindi 79.5 Hz potrebbe essere la terza armonica della componente a 26.5 Hz, non una seconda onda indipendente.

Il codice finale affronta questo problema usando una funzione che individua due picchi indipendenti e sopprime, dopo ogni selezione, le armoniche associate al picco già scelto. Questa logica è implementata in `find_two_independent_peaks()`.

1.8 Magnitude nell'intorno della frequenza

L'indicazione pratica alla base della parte finale del progetto è stata quella di calcolare la short-FFT e leggere la componente complessa nell'intorno della frequenza dell'onda quadra. La magnitude di tale componente viene usata come valore rappresentativo del potenziometro.

Il codice non legge un singolo bin FFT, perché sarebbe troppo fragile; dunque si calcola la magnitude in una piccola banda attorno alla frequenza target.

La funzione `band_magnitude()` fa questo:

```
1 mask = np.abs(freqs - center_freq) <= delta_hz
2
3 if not np.any(mask):
4     return 0.0
5
6 return np.sqrt(np.sum(amp[mask] ** 2))
```

La funzione `get_magnitude_at_freq()` richiama `positive_spectrum()` e poi `band_magnitude()`, restituendo la magnitude nell'intorno della frequenza scelta.

Matematicamente, se B è l'insieme dei bin compresi nell'intorno della frequenza f_0 , il codice calcola:

$$M(f_0) = \sqrt{\sum_{k \in B} |X[k]|^2}$$

dove B è definito da:

$$|f_k - f_0| \leq \Delta$$

Nel codice finale:

$$\Delta = 2.0 \text{ Hz}$$

1.9 Ricostruzione tramite box filter

Oltre alla stima delle magnitude, il progetto implementa anche una ricostruzione delle componenti principali del segnale.

L'idea è applicare un filtro passabanda nel dominio della frequenza:

1. si calcola la FFT del segnale;
2. si costruisce una maschera attorno alla frequenza di interesse f_0 ;
3. si azzerano tutti i coefficienti fuori dalla banda;
4. si esegue la IFFT;
5. si ottiene una componente ricostruita nel dominio del tempo.

La funzione `box_filter_reconstruct()` implementa questa logica:

```
1 x = np.asarray(buffer, dtype=float)
2 x = x - np.mean(x)
3
4 N = len(x)
5 X = fft(x)
6 f = fftfreq(N, 1 / fs)
7
8 mask = (np.abs(f - f0) <= delta_hz) | (np.abs(f + f0) <= delta_hz)
9
10 Xf = np.zeros_like(X)
11 Xf[mask] = X[mask]
12
13 y_rec = np.real(iff(Xf))
14 return y_rec
```

La maschera considera sia $+f_0$ sia $-f_0$, perché usando la FFT completa di un segnale reale lo spettro è simmetrico. Se si mantenesse solo la parte positiva, la ricostruzione nel tempo non sarebbe corretta.

La ricostruzione non produce necessariamente un'onda quadra perfetta. Se la banda è stretta e include principalmente la fondamentale, il segnale ricostruito sarà più simile a una sinusoide.

Capitolo 2

Implementazione e risoluzione dei problemi

2.1 Organizzazione del codice

La versione finale del progetto è divisa in più file:

```
1 script.py      → acquisizione live e coordinamento
2 fft.py         → funzioni di analisi spettrale e ricostruzione
3 plotting.py    → funzioni di visualizzazione
4 varie.py       → gestione di funzioni standard
5 Arduino sketch → lettura analogica e invio seriale
```

2.2 File script.py: flusso principale

Il file script.py è il cuore del programma. Si occupa di:

1. aprire la connessione seriale;
2. inizializzare i plot;
3. leggere continuamente i dati da Arduino;
4. gestire un buffer di campioni;
5. eseguire periodicamente la FFT;
6. individuare due componenti indipendenti;
7. calcolare le magnitude;
8. ricostruire le componenti tramite box filter;
9. aggiornare i grafici;
10. salvare i dati nel CSV.

I parametri principali sono:

```
1 PORT = "COM3"
2 BAUD = 115200
3 BUFFER_SIZE = 1500
4 TIMEOUT = 1
5
6 DELTA = 2.0
7
8 FS_FIXED = 2445.0
```

BUFFER_SIZE = 1500 indica che ogni analisi viene fatta su una finestra di 1500 campioni. FS_FIXED = 2445.0 è la frequenza di campionamento stimata sperimentalmente. DELTA = 2.0 è la larghezza della banda usata per la magnitude e per il box filter.

2.3 Stima della frequenza di campionamento

Una fase importante dello sviluppo ha riguardato la stima della frequenza di campionamento.

La frequenza di campionamento è stata stimata contando i campioni validi ricevuti in una finestra temporale nota, per esempio 10 secondi. I risultati ottenuti erano molto stabili:

```
1 25177 campioni in 10.317 s → 2440.43 Hz
2 25246 campioni in 10.316 s → 2447.17 Hz
3 25263 campioni in 10.316 s → 2448.89 Hz
```

Quindi è stato scelto:

```
1 FS_FIXED = 2445.0
```

Impostando f_s a 2445 Hz, le stesse componenti sono state correttamente scalate intorno a 18–20 Hz e 39–41 Hz, ovvero i valori attesi misurati sperimentalmente con l'oscilloscopio.

Infatti:

$$12 \cdot \frac{2445}{1500} \approx 19.56$$

e:

$$24 \cdot \frac{2445}{1500} \approx 39.12$$

2.3.1 Perché non usare i timestamp lato PC per ogni campione

Durante lo sviluppo è stata valutata la possibilità di stimare dinamicamente il sampling rate usando timestamp lato PC, per esempio con `time.time()`.

Il problema è che questo timestamp non rappresenta il momento in cui Arduino acquisisce il campione, ma rappresenta solo il momento in cui Python processa la riga.

A causa del buffering seriale, Python può ricevere molti campioni insieme. Se questi campioni vengono processati molto velocemente uno dopo l'altro, i timestamp lato PC risultano quasi identici, generando stime istantanee del sampling rate enormi o instabili.

Per questo motivo è stato preferito un valore fisso di f_s , stimato sperimentalmente su finestre temporali lunghe. Questa scelta è più stabile per l'analisi live.

2.4 File `fft.py`: calcolo dello spettro positivo

Nel file `fft.py`, la funzione di base è `positive_spectrum()`:

```
1 def positive_spectrum(buffer, fs):
2     x = np.asarray(buffer, dtype=float)
3
4     if len(x) == 0:
5         return None, None
6
7     x = x - np.mean(x)
8
9     N = len(x)
10    freqs = rfftfreq(N, d=1 / fs)
11    X = rfft(x)
12
13    amp = 2 * np.abs(X) / N
14
15    if len(amp) > 0:
16        amp[0] = 0
17
18    return freqs, amp
```

Questa funzione:

1. converte il buffer in un array NumPy;
2. controlla che non sia vuoto;
3. sottrae la media;
4. calcola la FFT reale;

5. costruisce l'asse delle frequenze positive;
6. normalizza l'ampiezza;
7. annulla la componente DC;
8. restituisce frequenze e ampiezze.

Questa funzione è la base per quasi tutta l'elaborazione successiva: calcolo delle magnitude, individuazione dei picchi, ricostruzione e analisi.

2.5 Magnitude nell'intorno della frequenza

La magnitude associata a un potenziometro viene calcolata in due passaggi.

Prima `band_magnitude()` seleziona i bin vicini alla frequenza scelta:

```
1 mask = np.abs(freqs - center_freq) <= delta_hz
```

Poi calcola:

```
1 return np.sqrt(np.sum(amp[mask] ** 2))
```

La funzione `get_magnitude_at_freq()` richiama `positive_spectrum()` e `band_magnitude()`.

La scelta di usare una banda invece di un singolo bin è importante perché la frequenza reale del segnale può non coincidere esattamente con un bin FFT. In quel caso l'energia si distribuisce sui bin vicini. Leggere un solo bin renderebbe il risultato instabile; leggere una piccola banda rende la stima più robusta.

Nel codice finale:

```
1 DELTA = 2.0
```

quindi la magnitude viene calcolata usando un intorno di ± 2 Hz attorno alla frequenza individuata.

2.6 Individuazione delle componenti principali

La funzione finale usata per individuare le due componenti principali è:

```
1 find_two_independent_peaks()
```

Questa funzione riceve:

```
1 buffer ,
2 fs ,
3 f_min=10.0 ,
4 f_max=120.0 ,
5 delta_hz=2.0 ,
6 min_sep_hz=5.0 ,
7 min_score_ratio=0.20
```

e restituisce due frequenze indipendenti.

La logica è:

1. calcola lo spettro positivo;
2. copia le ampiezze in `work_amp`;
3. cerca il picco massimo nella banda scelta;
4. salva il picco;
5. sopprime il picco scelto e le sue armoniche;
6. sopprime anche eventuali subarmoniche evidenti;
7. cerca un secondo picco nello spettro rimanente;
8. accetta il secondo picco solo se è abbastanza forte rispetto al primo.

La parte centrale è:

```
1 idx_candidates = np.where(mask)[0]
2 idx_peak = idx_candidates[np.argmax(work_amp[idx_candidates])]
3
4 f_peak = freqs[idx_peak]
5 peak_amp = work_amp[idx_peak]
```

Poi, se è già stato scelto un primo picco, il secondo viene accettato solo se ha score sufficiente:

```
1 if selected:
2     first_score = debug[0]["score"]
3     if peak_amp < first_score * min_score_ratio:
4         break
```

Questo evita di interpretare rumore o picchi molto deboli come una seconda componente reale.

2.6.1 Soppressione delle armoniche

Dopo aver selezionato un picco, il codice sopprime sia il picco stesso sia le sue armoniche:

```
1 for h in [1, 2, 3, 4, 5, 6, 7]:
2     fh = f_peak * h
3     if fh > f_max:
```

```
4         continue
5
6     suppress_mask = np.abs(freqs - fh) <= delta_hz
7     work_amp[suppress_mask] = 0
```

Anche se le onde quadre ideali hanno armoniche dispari, nel codice vengono sopresse anche armoniche pari. Questa scelta è pratica: il segnale reale può non essere una quadra ideale, può avere distorsioni, rumore, componenti spurie o effetti del circuito. Sopprimere anche multipli pari rende l'algoritmo più robusto nel contesto sperimentale.

Poi vengono sopresse anche subarmoniche evidenti:

```
1 for h in [2, 3, 4, 5, 6, 7]:
2     fh = f_peak / h
3     if fh < f_min:
4         continue
5
6     suppress_mask = np.abs(freqs - fh) <= delta_hz
7     work_amp[suppress_mask] = 0
```

Questa seconda soppressione serve a evitare che, se si seleziona un picco forte a 40 Hz, il codice selezioni poi 20 Hz solo perché è correlato come metà del primo picco.

Questa soluzione è dinamica: non hardcode le frequenze 18 Hz e 40 Hz. Il codice può quindi funzionare con altre coppie di frequenze, purché siano presenti nello spettro, sufficientemente separate e abbastanza forti.

2.6.2 Gestione delle finestre non affidabili

Il codice può trovarsi davanti a finestre in cui non riesce a individuare due componenti affidabili. Questo accade soprattutto durante i transitori, cioè mentre si ruotano i potenziometri.

Durante un transitorio, il segnale non è stazionario: la forma d'onda cambia all'interno della stessa finestra. La FFT assume invece che la finestra analizzata rappresenti un segnale relativamente stabile. Se il segnale cambia troppo durante la finestra, lo spettro diventa più confuso.

Per questo, se `find_two_independent_peaks()` restituisce `None`, il codice stampa:

```
1 print("[WARN] Non sono state trovate due componenti affidabili.\n")
2 continue
```

Invece di forzare un risultato, il programma ignora quella finestra e continua.

2.7 Ricostruzione delle componenti

Dopo aver individuato f_1 e f_2 , il codice calcola:

```
1 y1 = box_filter_reconstruct(  
2     sig,  
3     sample_rate,  
4     f0=f1,  
5     delta_hz=DELTA  
6 )  
7  
8 y2 = box_filter_reconstruct(  
9     sig,  
10    sample_rate,  
11    f0=f2,  
12    delta_hz=DELTA  
13 )
```

Queste due ricostruzioni rappresentano le componenti del segnale associate alle due frequenze principali. Il risultato viene poi passato al plot:

```
1 update_reconstruct_plot(  
2     ax_rec,  
3     line_sig_1,  
4     line_sig_2,  
5     y1,  
6     y2,  
7     f1,  
8     f2  
9 )
```

In questo modo il programma non si limita a stampare le frequenze e le magnitude, ma mostra anche graficamente le due componenti filtrate nel tempo.

2.8 File plotting.py

Il file `plotting.py` contiene le funzioni di visualizzazione.

La funzione `setup_live_plot()` crea il grafico del segnale acquisito in tempo reale:

```
1 ax.set_ylim(0, 1023)  
2 ax.set_xlabel("Campioni (ultimi N)")  
3 ax.set_ylabel("Valore analogico")  
4 ax.set_title("Lettura in tempo reale")
```

La funzione `update_live_plot()` aggiorna il grafico usando i dati presenti nei buffer.

La funzione `setup_reconstruct_plot()` crea invece il grafico delle due componenti ricostruite:

```
1 ax.set_title("Segnali separati (box filter in FFT)")
2 ax.set_xlabel("Campioni")
3 ax.set_ylabel("Valore ricostruito")
```

La funzione `update_reconstruct_plot()` aggiorna le curve `line_sig_1` e `line_sig_2`, modificando anche le label per indicare la frequenza associata alla componente:

```
1 line_sig_1.set_label(f"Componente ~ {f1:.1f} Hz")
2 line_sig_2.set_label(f"Componente ~ {f2:.1f} Hz")
```

Questo permette di vedere in tempo reale quali componenti sono state separate dal box filter.

2.9 File `varie.py`

Il file `varie.py` contiene funzioni di utilità.

La funzione `open_serial()` apre la porta seriale:

```
1 ser = serial.Serial(port, baud, timeout=timeout)
2 time.sleep(2)
3 ser.reset_input_buffer()
4 return ser
```

La pausa iniziale permette ad Arduino di stabilizzarsi dopo l'apertura della seriale. Il reset del buffer evita di analizzare dati vecchi già presenti nella seriale.

La funzione `save_csv()` salva un file CSV contenente:

- indice del campione;
- valore originale;
- valore ricostruito;
- frequenza associata.

La funzione `setup_exit()` gestisce l'interruzione da tastiera, chiudendo i grafici in modo più ordinato.

Capitolo 3

Testing e risultati ottenuti

3.1 Validazione su segnali simulati

Prima di testare il circuito reale, è stata usata una simulazione con due onde quadre note.

Il test generava due onde quadre con frequenze:

```
1 12.5 Hz
2 26.5 Hz
```

e ampiezze diverse. Lo scopo era verificare che l'algoritmo fosse in grado di riconoscere le due componenti reali senza confondere le armoniche.

Dopo la correzione dell'algoritmo, il test ha restituito:

```
1 Fondamentali attese: 12.5 26.5
2 Fondamentali trovate: 12.5 26.5
```

Questo ha mostrato che l'algoritmo funziona correttamente in un caso controllato.

È stato poi fatto un test con una sola onda quadra a 26.5 Hz. Il comportamento desiderato era evitare che l'algoritmo inventasse una seconda componente forte partendo dalle armoniche della prima. Dopo l'aggiunta della soglia `min_score_ratio`, il test ha restituito:

```
1 Fondamentali trovate: None None
```

con debug che mostrava una prima componente forte e una seconda molto debole. Questo ha confermato che la soglia elimina falsi positivi dovuti a rumore o armoniche deboli.

3.2 Test live con potenziometri

Nel test live è stato eseguito il seguente esperimento:

1. fase iniziale stabile;
2. aumento del primo potenziometro;
3. ritorno del primo potenziometro;
4. aumento del secondo potenziometro;
5. ritorno del secondo potenziometro.

Durante la fase stabile iniziale, il codice ha trovato ripetutamente componenti intorno a:

```
1 17.93 Hz
2 40.75 Hz
```

con magnitude circa:

```
1 Magnitude potenziometro 1: ~88
2 Magnitude potenziometro 2: ~90-94
```

Nel log si vede una sequenza stabile di misure con frequenze 17.93 Hz e 40.75 Hz e magnitude simili, per esempio 87.87/93.76, 88.31/92.73, 88.22/91.33. Questo indica che, a manopole ferme, il sistema produce valori ripetibili.

Anche durante le fasi di modifica dei potenziometri sono stati forniti risultati compatibili con i valori attesi.

3.3 Risultato finale

Il risultato finale del progetto è un sistema in grado di:

- acquisire un segnale analogico misto tramite Arduino;
- trasferire i dati al PC via seriale;
- raccogliere i campioni in finestre da 1500 campioni;
- calcolare una short-FFT su ogni finestra;
- individuare dinamicamente due picchi indipendenti nello spettro;
- evitare di confondere armoniche e segnali indipendenti;
- calcolare la magnitude delle componenti spettrali;
- usare tali magnitude come valori rappresentativi/proporzionali dei due potenziometri;
- ricostruire nel tempo le componenti principali tramite box filter;
- visualizzare sia il segnale acquisito sia le componenti ricostruite.